



IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 13 - Coherencia de Caché

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), José Mendoza (jfmendoza@uc.cl)

Pregunta 1: Preguntas conceptuales

- (a) [P4(d) - I3 2024'1] Una variación del protocolo MESI de coherencia de caché es el protocolo MSI (sin el estado *Exclusive*). ¿Qué ventaja tiene tener el estado *Exclusive*? Y ¿cómo se hace cargo el protocolo MSI de no tener el estado *Exclusive*? Responde de manera clara, precisa y breve.

Solución: El estado *Exclusive* permite reducir el tráfico en el bus compartido por todas las memorias caché en caso de que el procesador realice una solicitud de escritura sobre una línea en este estado. Esto se debe a que como solo su caché posee la copia del bloque de memoria, no es necesario invalidar copias de otras caché. En el protocolo MSI, en cambio, se hace uso del estado *S* (*Shared*) aunque **una** sola caché posea una copia de un bloque de memoria. Por lo tanto, si se realizan solicitudes de escritura sobre ella, se emitirá la señal correspondiente para invalidar otras copias, a pesar de que no exista ninguna.

- (b) En arquitecturas multiprocesador con memoria compartida, utilizar una política de escritura *write-through* en las memorias caché evitaría problemas graves de coherencia. Si esto es así, ¿por qué los sistemas modernos optan por utilizar protocolos complejos de coherencia basados en la política *write-back*? Responde de manera concisa.

Solución: Aunque *write-through* mantendría la memoria consistente, genera un *overhead* significativo, ya que cada operación de escritura debe ir hacia la memoria principal, aumentando drásticamente el tráfico y el tiempo de ejecución de los programas. Por ello, se prefiere *write-back*, el cual reduce el tráfico al escribir en memoria principal solo cuando se reemplaza el bloque, asumiendo a cambio el costo de tener que implementar los protocolos de coherencia de caché.

Pregunta 2: Coherencia caché [P6(d) - Examen 2023'1]

Suponga que posee una arquitectura MIMD de memoria compartida con 2 CPU que poseen su propia caché y que siguen el protocolo MESI para asegurar consistencia. La memoria contiene los siguientes datos de variables y las CPU0 y CPU1 ejecutan los siguientes programas:

Dirección	Label	Valor
0x00	i	1
0x01	arr	1
0x02		10

```
// CPU0      // CPU1
MOV B,(i)     MOV A,(arr)
MOV A,(arr)   MOV B,arr
ADD B,A       ADD B,A
SHR A,A       MOV (B),A
MOV (B),A     NOP
```

Asumiendo que cada dirección se almacena en una línea distinta, indique el estado de cada línea de la caché (M/E/S/I) para cada iteración completando la tabla adjunta, asumiendo que todas las líneas parten en estado I:

Instrucción	CPU0			CPU1		
	i	arr[0]	arr[1]	i	arr[0]	arr[1]
1						
2						
3						
4						
5						

Solución: A continuación se muestra la tabla final esperada, que incluye columnas para los valores de los registros A y B y el valor en memoria de las variables en cada iteración:

Instrucción	CPU0					CPU1					Memoria		
	i	arr[0]	arr[1]	A	B	i	arr[0]	arr[1]	A	B	i	arr[0]	arr[1]
1	E	I	I	-	1	I	E	I	1	-	1	1	10
2	E	S	I	1	1	I	S	I	1	1	1	1	10
3	E	S	I	1	2	I	S	I	1	2	1	1	10
4	E	S	I	0	2	I	S	M	1	2	1	1	1
5	E	S	M	0	2	I	S	I	1	2	1	1	0

Pregunta 3: Coherencia caché [P2 - I3 2025'2]

Suponga que posee una arquitectura MIMD de memoria compartida con 3 CPU que poseen su propia caché y que siguen el protocolo MESI para asegurar consistencia. La memoria contiene los siguientes datos de variables; mientras que CPU0, CPU1 y CPU2 ejecutan los siguientes programas:

Dirección	Label	Valor
0x00	var	0
...	...	...
0xFD		0
0xFE		0
0xFF		0

// CPU0	// CPU1	// CPU2
MOV B, 1	MOV B, 253	MOV A, 1
PUSH B	MOV (var), B	MOV B, 2
MOV A, (var)	PUSH B	POP A
ADD A, 1	POP B	
MOV B, A		PUSH B
MOV (B), B	MOV B, (B)	PUSH B

Asumiendo que cada dirección se almacena en una línea distinta, y que el contador SP posee un valor inicial igual a 255 en **todas** las CPU, indique el estado de cada línea de las caché (M/E/S/I) para cada ciclo completando la tabla adjunta, asumiendo que todas las líneas parten en estado I (ciclo 0):

	CPU0				CPU1				CPU2			
Ciclo	0x00	0xFD	0xFE	0xFF	0x00	0xFD	0xFE	0xFF	0x00	0xFD	0xFE	0xFF
0	I	I	I	I	I	I	I	I	I	I	I	I
1												
2												
3												
4												
5												
6												

Solución: A continuación se muestra la tabla final esperada por cada iteración:

	CPU0				CPU1				CPU2			
Ciclo	0x00	0xFD	0xFE	0xFF	0x00	0xFD	0xFE	0xFF	0x00	0xFD	0xFE	0xFF
0	I	I	I	I	I	I	I	I	I	I	I	I
1	I	I	I	I	I	I	I	I	I	I	I	I
2	I	I	I	M	M	I	I	I	I	I	I	I
3	S	I	I	I	S	I	I	M	I	I	I	I
4	S	I	I	I	S	I	I	M	S	I	I	I
5	I	I	I	I	I	I	I	M	M	I	I	I
6	I	I	M	I	I	E	I	I	M	I	I	M

Para llegar al estado final correcto, es importante haber tomado en cuenta lo siguiente:

- Al ser SP parte de una CPU, su valor es **independiente** de lo que ejecuten otras. Es decir, que una CPU realice un PUSH o un POP no afecta el valor de SP de otra.
- La ejecución de POP A y POP B toma dos ciclos. La lectura del valor de memoria se realiza en el **segundo ciclo**, dado que el primero lo único que hace es incrementar el valor de SP.

- Al hacer POP sin haber hecho un PUSH antes, la operación generará un *overflow* sobre el contador SP, causando que su valor cambie de 255 a 0. Esto implica que, en la CPU2, al ejecutar POP realizará una lectura de memoria sobre la dirección 0 en el cuarto ciclo, *i.e.* la variable `var`.
- Si una CPU hace una lectura sobre una línea ya modificada por ella misma, esta sigue estando en dicho estado y solo cambia en dos escenarios: (1) si otra CPU realiza una lectura sobre ella, cambiando a estado *Shared*; o (2) si otra CPU realiza una escritura sobre ella, cambiando a estado *Invalid*.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

